

EARTHQUAKE PREDICTION MODEL USING PYTHON

Phase-5 Documentation Submission

Team Leader:

Kishore M

422521104017



Introduction:

- This project presents the development of an earthquake prediction model using Python.
- The primary objective is to leverage machine learning techniques to predict earthquake magnitudes based on a dataset obtained from Kaggle.
- A neural network model is designed and trained using TensorFlow or Keras to make predictions on earthquake magnitudes.
- Earthquakes are natural disasters that can cause significant damage and loss of life, making their accurate prediction a matter of utmost importance for public safety and disaster preparedness.
- This project presents the development of an earthquake prediction model using Python.

Data Source:

- The data is provided by the United States Geological Survey (USGS), which monitors and reports on earthquake activity worldwide.
- The dataset contains information about worldwide earthquake events, including location, time, magnitude, depth, and more. It is a comprehensive collection of earthquake-related data.

Dataset Link:

<https://www.kaggle.com/datasets/usgs/earthquake-database>

Innovation

Reason for choosing this Model:

Feature Learning: LSTMs can automatically learn and extract relevant features from raw seismic data.

Continual Learning: LSTMs can be trained in an online or continual learning fashion, allowing the model to adapt to new data and evolving seismic patterns over time.

Memory of Past Information: LSTMs have a built-in mechanism to remember past information while processing new data points.

Innovative Techniques :

- **Ensemble Method** is applied LSTM model to improve their predictive performance.
- Training multiple LSTM models with variations in architecture, hyperparameters. Each LSTM model should be trained independently on the same dataset.
- Using Bagging combine all the predictions from all individual LSTM models. The result is the final ensemble prediction.

Hyperparameter Tuning:

Activation function : relu, sigmoid, linear, sigmoid

Optimizer : adam, SGD,

Loss Function: mean_squared_error,

Batch size : 10, 16, 32

Epochs : 10,20,50

Dropout : 10% , 20%

Data Preprocessing

- Data preprocessing is a crucial step in machine learning. It involves cleaning, transforming, and organizing raw data into a format that is suitable for the training model.
- The primary goals of data preprocessing are to improve data quality, reduce NaN, and make the data more amenable to training a model.

Steps to Preprocessing Earthquake Dataset:

1. Check the Longitude and Latitude degree
2. Select the relevant column from the dataset
3. Convert the date and time into the same format
4. Convert date and time into TimeStamp
5. Remove NaN row or data from a dataset
6. Check for magnitude max and min (2.5 to 9.1)

1. Checking for maximum and minimum of Latitude and Longitude

- Always the Latitude and Longitude lies between constant range.
- Latitude should be between -90 to 90 degrees.
- Longitude should be lies between -180 to 180 degrees.

```
import pandas as pd

# Check latitude values between -90 to 90
df[df['Latitude'].between(-90, 90)]

# Check longitude values between -180 to 180
df[df['Longitude'].between(-180, 180)]

df.tail()
```

The `.between(-90,90)` method is used to check whether the value of latitude is falls between range or not. It returns a boolean result as 'True' or 'False'.

It filters the dataset, retaining only the rows for the condition 'True'. In other words, it keeps only the rows with valid latitude and longitude values

2. Select the relevant column from the dataset

- Remove the non-relevant column from the dataset
- Because it does not help the model to learn but instead makes the learning as complex.

```
[31]: data = data[['Date', 'Time', 'Longitude', 'Latitude', 'Depth', 'Magnitude']]
      data
```

```
[31]:
```

	Date	Time	Longitude	Latitude	Depth	Magnitude
0	01/02/1965	13:44:18	145.6160	19.2460	131.60	6.0
1	01/04/1965	11:29:49	127.3520	1.8630	80.00	5.8
2	01/05/1965	18:05:58	-173.9720	-20.5790	20.00	6.2
3	01/08/1965	18:49:43	-23.5570	-59.0760	15.00	5.8
4	01/09/1965	13:32:50	126.4270	11.9380	15.00	5.8
...	...	...	...	...	...	...
23407	12/28/2016	08:22:12	-118.8941	38.3917	12.30	5.6
23408	12/28/2016	09:13:47	-118.8957	38.3777	8.80	5.5
23409	12/28/2016	12:38:51	140.4262	36.9179	10.00	5.9
23410	12/29/2016	22:30:19	118.6639	-9.0283	79.00	6.3
23411	12/30/2016	20:08:28	141.4103	37.3973	11.94	5.5

23412 rows × 6 columns

3. Convert date and time into same format

Dataset contains date different format like dd-mm-yyyy, dd/mm/yyyy.

So, convert date and time into same format using datetime library.

```
import pandas as pd

df = pd.read_csv('database.csv')

def formatconvert(date_str):
    return pd.to_datetime(date_str).strftime('%d/%m/%Y')

df['Date'] = df['Date'].apply(formatconvert)

df['Date']
```

0	02/01/1965
1	04/01/1965
2	05/01/1965
3	08/01/1965
4	09/01/1965
...	
23406	28/12/2016
23407	28/12/2016
23408	28/12/2016
23409	29/12/2016
23410	30/12/2016

Name: Date, Length: 23411, dtype: object

4.Remove nan row or data from dataset

- Check for Not a Number in the given dataset. If any nan present in dataset, remove the row from the dataset.
- To remove rows or data with NaN values from a dataset in python
- We use dropna() from pandas. it returns a Boolean result

```
[60]: # Count the number of missing (NaN) values in each column
nan_counts = data.isna().sum()

# Print the results
print(nan_counts)
```

Latitude	0
Longitude	0
Depth	0
Magnitude	0
Timestamp	0

dtype: int64

5. Convert date and time into TimeStamp

Timestamps are numerical values representing a specific point in time, usually in seconds or milliseconds. It provide a consistent and standardized way to represent time across different systems and programming language.

The timestamps represent the number of seconds since the Unix epoch from January 1, 1970, to till date so, drop the before it.

```
[39]: df=pd.DataFrame(df)

df = df.drop(df.index[0:1457])
df.reset_index(drop=True, inplace=True)
df
```

```
[39]:
```

	Date	Time	Latitude	Longitude	Depth	Magnitude
0	01/04/1970	17:00:41	24.1850	102.5430	11.30	7.1
1	01/05/1970	11:49:10	23.9840	102.7320	15.00	5.9
2	01/06/1970	5:35:54	-9.5830	151.4930	15.00	6.3
3	01/07/1970	7:56:14	15.7850	-59.8080	36.70	6.0
4	01/08/1970	17:12:41	-34.8500	178.7820	199.40	6.8
...	...	...	...	...	...	...
21949	12/28/2016	8:22:12	38.3917	-118.8941	12.30	5.6
21950	12/28/2016	9:13:47	38.3777	-118.8957	8.80	5.5
21951	12/28/2016	12:38:51	36.9179	140.4262	10.00	5.9

- We can create a new column by combining the 'Date' and 'Time' Column into single timestamp using `pd.to_datetime.strptime()`
- Using `mktime()` we can convert date and time into timestamp.
- Finally remove the date and time column from the dataset.

```
[56]: import datetime
import time

timestamp = []
for d, t in zip(df['Date'], df['Time']):
    ts = datetime.datetime.strptime(d+' '+t, '%d/%m/%Y %H:%M:%S')
    timestamp.append(time.mktime(ts.timetuple()))

timeStamp = pd.Series(timestamp)

[57]: df['Timestamp'] = timeStamp.values
data = df.drop(['Date', 'Time'], axis=1)
data = data[data.Timestamp != 'ValueError']
data
```

```
[57]:
```

	Latitude	Longitude	Depth	Magnitude	Timestamp
0	24.1850	102.5430	11.30	7.1	3.006410e+05
1	23.9840	102.7320	15.00	5.9	3.683500e+05
2	-9.5830	151.4930	15.00	6.3	4.323540e+05
3	15.7850	-59.8080	36.70	6.0	5.271740e+05
4	-34.8500	178.7820	199.40	6.8	6.469610e+05
...	...	...	...	...	...
21949	38.3917	-118.8941	12.30	5.6	1.482894e+09
21950	38.3777	-118.8957	8.80	5.5	1.482897e+09
21951	36.9179	140.4262	10.00	5.9	1.482909e+09

6. Check for magnitude max and min (2.5 to 9.1)

Find Maximum and Minimum value for magnitude because the magnitude value cannot be negative or more than 10.

Check for magnitude value within 2.5 to 9.1, if anything exceeds delete the row or data from the dataset.


```
[68]: data['Magnitude'].max()
```

```
[68]: 9.1
```

```
[67]: data['Magnitude'].min()
```

```
[67]: 5.5
```

```
[69]: data[data['Magnitude'].between(5.5, 9.1)]
```

```
[69]:
```

	Latitude	Longitude	Depth	Magnitude	Timestamp
0	24.1850	102.5430	11.30	7.1	3.006410e+05
1	23.9840	102.7320	15.00	5.9	3.683500e+05
2	-9.5830	151.4930	15.00	6.3	4.323540e+05
3	15.7850	-59.8080	36.70	6.0	5.271740e+05
4	-34.8500	178.7820	199.40	6.8	6.469610e+05
...	...	...	...	...	...
21949	38.3917	-118.8941	12.30	5.6	1.482894e+09
21950	38.3777	-118.8957	8.80	5.5	1.482897e+09
21951	36.9179	140.4262	10.00	5.9	1.482909e+09
21952	-9.0283	118.6639	79.00	6.3	1.483031e+09
21953	37.3973	141.4103	11.94	5.5	1.483109e+09

```
21954 rows x 5 columns
```

7.Display the data after preprocessing

Dataset after preprocessing is done. Save the file in csv format using `to_csv()`.

The preprocessed dataset is use for train the model.

```
[72]: data.to_csv("resultdata.csv", index=False)  
data
```

```
[72]:
```

	Latitude	Longitude	Depth	Magnitude	Timestamp
0	24.1850	102.5430	11.30	7.1	3.006410e+05
1	23.9840	102.7320	15.00	5.9	3.683500e+05
2	-9.5830	151.4930	15.00	6.3	4.323540e+05
3	15.7850	-59.8080	36.70	6.0	5.271740e+05
4	-34.8500	178.7820	199.40	6.8	6.469610e+05
...	...	...	...	...	...
21949	38.3917	-118.8941	12.30	5.6	1.482894e+09
21950	38.3777	-118.8957	8.80	5.5	1.482897e+09
21951	36.9179	140.4262	10.00	5.9	1.482909e+09
21952	-9.0283	118.6639	79.00	6.3	1.483031e+09
21953	37.3973	141.4103	11.94	5.5	1.483109e+09

21954 rows × 5 columns

Result of Preprocessing:

- The preprocessing steps are foundational in preparing the earthquake dataset for further analysis and model development.
- By cleaning, organizing, and standardizing the data, we create a solid foundation for accurate and meaningful predictions regarding earthquake occurrences and magnitudes.
- This dataset can now be used for exploratory data analysis, feature engineering, and the development of machine learning models.

As we progress further, we will dive into more advanced aspects of machine learning and develop a model that can potentially contribute to earthquake forecasting and risk reduction

Model Creation

Long Short-Term Memory Neural Networks

LSTM networks are a type of recurrent neural network (RNN) capable of modeling temporal dependencies in sequential data

LSTMs address this problem by introducing a memory cell, which is a container that can hold information for an extended period. The memory cell is controlled by three gates: the input gate, the forget gate, and the output gate.

Create LSTM:

Create a Sequential model using Keras and then add layers one by one and data flows sequentially through these layers.

Define the LSTM layer with 64 units. It represents the number of LSTM units or neurons.

Input shape: represents no time step in the input sequence. I mention time steps in the model.

Compilation:

we compile the model by specifying the optimizer and loss function

Optimizer: Adam is chosen for efficient weight updates during training.

Loss Function: Mean Squared Error, minimize the squared difference between predicted and actual values.

Dropout Layer:

we are adding a dropout layer to the model with a dropout rate of 0.2. at approximately 20% of neurons in the previous layer will be randomly “dropped out “ or deactivated.

Fully-Connected Layer:

The output layer has a single neuron. in regression tasks where you want to predict a single continuous value, such as earthquake magnitude.

Activation: ‘Linear’ function means that the output of the neuron is directly proportional to the weighted sum of its inputs

Model Training:

we train the model on the training data for 50 epochs with a batch size of 32. During training, the model learns to map the input features to earthquake magnitudes. Validation data is used to monitor the model's performance during training.

Model Building

Import Modules:

Import statements provide the necessary tools to build, train, and evaluate your deep learning model for earthquake magnitude prediction. The model architecture includes LSTM layers and fully connected (Dense) layers, which are common components in many sequential models for regression tasks.

Data Loading and Preprocessing:

The code loads your earthquake data from a CSV file named 'resultdata.csv' using `pd.read_csv`.

Feature Extraction:

It extracts features from the dataset, which include 'Longitude', 'Latitude', 'Depth', and 'Timestamp'. These features are stored in the features array.

The target variable, 'Magnitude', is stored in the labels array.

Normalization:

The longitude and latitude features are normalized to a range between 0 and 1 using the `MinMaxScaler` from `scikit-learn`. This scaling helps neural networks perform better.

Train-Test Split:

The dataset is split into training and testing sets using `train_test_split` from `scikit-learn`. The split ratio is 80% for training and 20% for testing.

LSTM Model:

- Input shape: represents no of time step in the input sequence. I mention time steps in the model.
- Dropout Layer: we are adding a dropout layer to the model with a dropout rate of 0.2. at approximately 20% of neurons in the previous layer will be randomly “dropped out “ or deactivated
- Fully-Connected Layer: The output layer has a single neuron. in regression tasks where you want to predict a single continuous value, such as earthquake magnitude.
- Activation: ‘Linear’ function means that the output of the neuron is directly proportional to the weighted sum of its inputs.

```
model = Sequential()
model.add(LSTM(64, input_shape=(X_train.shape[1], 1),
return_sequences=True)) model.add(LSTM(64))
model.add(Dense(32,activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(1, activation='linear'))
```

Model Compilation:

The model is compiled with the 'adam' optimizer and the 'mean_squared_error' loss function. This is a regression problem, so the mean squared error is an appropriate loss function.

Model Training:

The model is trained using the training data with 10 epochs and a batch size of 32. It also validates the model's performance on the test data during training.

Model Evaluation:

The model's performance is evaluated on the test set, and the test loss (mean squared error) is printed to the console.

Loss Function: Mean Squared Error, minimize the squared difference between predicted and actual values.

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

Making Predictions:

The model is used to make predictions on the test data, and the predicted earthquake magnitudes are stored in the predictions array.

Train the model and predict the result using the test dataset to calculate the result we use Mean Square Error.

```
[16]: import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

# Load your earthquake data (replace 'your_data.csv' with your dataset)
data = pd.read_csv('resultdata.csv')

# Assuming your dataset has columns 'Longitude', 'Latitude', and 'magnitude'
# Adjust this accordingly based on your actual data columns
features = data[['Longitude', 'Latitude', 'Depth', 'timestamp']].values
labels = data['Magnitude'].values

# Normalize Longitude and Latitude features to a range between 0 and 1
scaler = MinMaxScaler()
scaled_features = scaler.fit_transform(features)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    scaled_features, labels, test_size=0.2, random_state=42)
X_train = np.asarray(X_train).astype(np.float32)
y_train = np.asarray(y_train).astype(np.float32)
X_test = np.asarray(X_test).astype(np.float32)
y_test = np.asarray(y_test).astype(np.float32)

# Build the LSTM model
model = Sequential()
model.add(LSTM(64, input_shape=(X_train.shape[1], 1), return_sequences=True))
model.add(keras.layers.LSTM(units=64))
model.add(Dense(32, activation='linear'))
model.add(Dense(32, activation='linear'))
model.add(Dropout(0.2))

model.add(Dense(1, activation='linear')) # Output Layer for predicting magnitude

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Train the model
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))

# Evaluate the model on the test set
loss = model.evaluate(X_test, y_test)
print(f'Test Loss: {loss}')

# Make predictions
predictions = model.predict(X_test)

# You can now use predictions for further analysis or visualization
```

Result:

The result of the program is a quantitative evaluation of the earthquake magnitude prediction model. the test loss is reported as 0.1799456. This value represents the mean squared error (MSE) of the model's predictions on the test dataset.

Test Loss: 0.178866684436

A test loss of 0.178866684436 indicates the average squared difference between the model's predicted earthquake magnitudes and the actual earthquake magnitudes in the test dataset. A lower test loss is generally preferred, as it implies that the model's predictions are closer to the actual values.

Keep in mind that while the test loss is an essential quantitative metric, it's often a good practice to complement it with visualizations and other evaluation metrics to gain a more comprehensive understanding of the model's performance.

The interpretation of the test loss value can also depend on the specific context and requirements of your earthquake magnitude prediction project.

▼ Model Evaluation:

- The model evaluation is done by using evaluate()
- Evaluation is compared with predicted data with actual result

```
[27]: # Evaluate the model on the test set
      loss = model.evaluate(X_test, y_test)
      print(f'Test Loss: {loss}')

138/138 [=====] - 1s 5ms/step - loss: 0.1789
Test Loss: 0.1788666844367981

[ ]:
```

Conclusion

- They empower individuals, organizations, and communities to better understand seismic risks and make informed decisions to mitigate the impact of earthquakes.
- By combining data analysis, machine learning, and geospatial visualization, we enhance our ability to comprehend, predict, and respond to seismic events.
- In conclusion, this project is a starting point for earthquake magnitude prediction using deep learning. It demonstrates the potential of LSTM models in capturing temporal patterns in earthquake data. Further refinements and research are required to improve the accuracy and reliability of earthquake predictions, contributing to the safety and preparedness of regions prone to seismic activity.